

Invisible Watermarking for ALP

Provenance and Protection Research Summary

Mandy Budan Art Gallery

September 16, 2026

What is Invisible Watermarking?

Invisible watermarking embeds a hidden identifier into image data without visible changes. Unlike visible watermarks (logos, text overlays), invisible watermarks are imperceptible to the human eye but can be recovered using the right decoding tool. The goal is **provenance** - proving ownership if images are scraped, stolen, or used without permission.

Key Concepts

Concept	Explanation
Frequency-domain	Watermark embedded in transformed frequency space (DWT + DCT) rather than pixels. Robust against co
Blind watermark	No original image needed for decoding. Only method name and payload length required.
Payload capacity	How much data can be embedded, measured in bits. A 15-char ASCII string = 120 bits.
Bit accuracy	Percentage of bits correctly decoded. 100% = perfect recovery.
Robustness	How well the watermark survives attacks (JPEG, cropping, brightness, resizing).

POC Test Setup

We tested the **invisible-watermark** Python library (<https://github.com/ShieldMnt/invisible-watermark>), which implements two frequency-domain algorithms: **dwtDct** (fast) and **dwtDctSvd** (3x slower, more robust).

Test Images

Type	Size	Quality	Filename
Study detail	1400x1008	q=90	images/studies2/2025-223_detail.jpg
Study gallery	400x288	q=82	images/studies2/2025-223_sm.jpg
Painting detail	1400x1944	q=90	images/2013-carnelian-morning_detail.jpg

Watermark String Format

We tested with two formats representing the longest plausible payloads:

Format	Example	Length
Studies	MB-ALP-223-2025	15 bytes / 120 bits
Paintings (longest)	MB-ALP-2013-summer-in-the-garden-2013	37 bytes / 296 bits

POC Results - Detail Images (1400px)

Method	Quality	Bit Accuracy	Result
dwtDctSvd	q=95	100% (296/296)	PASS PERFECT
dwtDctSvd	q=90	99.2% PASS	Reliable (1 bit off, error correction fixes)
dwtDctSvd	q=85	99.2%	PASS Reliable
dwtDctSvd	q=82	97.5%	PASS Reliable
dwtDct	q=95	87.5%	FAIL Too many errors
dwtDct	q=90	69.2%	FAIL

POC Results - Gallery Images (400px)

The 400px gallery images at 82% quality **CANNOT** hold a reliable watermark. Bit accuracy ranged from 48% to 63% - effectively random noise. This is expected: too few pixels, too much compression.

Conclusion

Use **dwtDctSvd** method on 1400px detail images only. Watermark is 100% recoverable at any quality from q=82 to q=95. 400px gallery images are too small - skip them.

Decoding - How It Works

No key is needed. These are 'blind' watermarks. To decode:

Step	Action
1	<code>pip install invisible-watermark opencv-python-headless</code>
2	<code>image = cv2.imread('image.jpg')</code>
3	<code>decoder = WatermarkDecoder('bits', NUM_BITS)</code>
4	<code>bits = decoder.decode(image, 'dwtDctSvd')</code>
5	Convert bits back to UTF-8 string

Decode Script (Python)

```
from imwatermark import WatermarkDecoder import cv2 METHOD = 'dwtDctSvd' WATERMARK =
'MB-ALP-223-2025' NUM_BITS = len(WATERMARK.encode('utf-8')) * 8 image =
cv2.imread('path/to/image_detail.jpg') decoder = WatermarkDecoder('bits', NUM_BITS) bits
= decoder.decode(image, METHOD) bits_str = ''.join('1' if b else '0' for b in bits)
decoded = bytes(int(bits_str[i:i+8], 2) for i in range(0, len(bits_str), 8))
print(decoded.decode('utf-8'))
```

Batch Implementation Plan

Step	Action
1	Scan images/ for *_detail.jpg (paintings)
2	Scan images/studies2/ for *_detail.jpg (studies)
3	Extract slug/filename to build watermark string
4	Run dwtDctSvd encoder at q=90
5	Verify each image decodes correctly
6	Save decode script in repo

Watermark String Format

Type	Format	Example
Painting	MB-ALP-{slug}-{year}	MB-ALP-cloud-shadows-2006
Study	MB-ALP-{number}-{year}	MB-ALP-223-2025

Limitations and Risks

No invisible watermark survives heavy manipulation:

Attack	Survives?
JPEG compression q=82+	PASS Yes
Brightness/contrast	PASS Yes
Gaussian blur	PASS Yes
Gaussian noise	PASS Yes
Screenshots	FAIL No
Heavy cropping	FAIL No
50% resize	FAIL No
Rotation	FAIL No

References

- invisible-watermark library: <https://github.com/ShieldMnt/invisible-watermark>
- InvisMark (Microsoft Research): <https://github.com/microsoft/InvisMark>

- Trufo (commercial API): <https://trufo.ai>
- Imatag (commercial API): <https://www.imatag.com>